

Laravel Project - README

Introduction

This is a basic Laravel project setup. This file explains the usage of **Artisan** commands, provides a cheatsheet for the most commonly used commands, and gives an overview of the **Laravel architecture, structure, design patterns, and technologies** Laravel uses.

Artisan Overview

Artisan is the command-line interface (CLI) included with Laravel. It provides a number of helpful commands for common tasks such as database migrations, generating code, and running tests.

Common Artisan Commands Cheatsheet

1. **Create a new Laravel project**

```
composer create-project --prefer-dist laravel/laravel example-app
```

2. **Make a migration**

Create a new database migration file.

```
php artisan make:migration create_users_table
```

3. **Run migrations**

Run all the migrations to create/update tables.

```
php artisan migrate
```

4. **Rollback the last migration**

Revert the last migration (useful for undoing changes).

```
php artisan migrate:rollback
```

5. **Generate a controller**

Create a new controller.

```
php artisan make:controller UserController
```

6. **Generate a model**

Create a new Eloquent model.

```
php artisan make:model User
```

7. **Serve the application**

Run Laravel's built-in development server.

```
php artisan serve
```

8. **Clear the application cache**
Clear the application's cached data.

```
php artisan cache:clear
```
 9. **List all available commands**
Display all Artisan commands.

```
php artisan list
```
 10. **Create a new migration for adding a column to a table**

```
sh php artisan make:migration add_column_to_users_table  
--table=users
```
-

Laravel Architecture & Structure

Laravel follows a **Model-View-Controller (MVC)** architecture, which separates the logic into three main components:

1. **Model** - Handles the interaction with the database. In Laravel, Eloquent is used to define the models and relationships.
Example: `User`, `Post`, `Product`.
 2. **View** - Displays the user interface. Laravel uses **Blade**, a templating engine, to render views.
Example: `home.blade.php`, `product.blade.php`.
 3. **Controller** - Manages the application's logic and user input. It interacts with the Model and passes data to the View.
Example: `UserController`, `ProductController`.
-

Directory Structure

- **app/**
This is where the core of your application lives. It contains:
 - **Console/**: Custom Artisan commands.
 - **Http/**: Controllers, middleware, and requests.
 - **Models/**: Eloquent models.
 - **Providers/**: Service providers that bind services into the container.
- **bootstrap/**
Contains the `app.php` file which is responsible for bootstrapping the application.
- **config/**
Holds all your application configuration files (e.g., `database.php`, `app.php`, `mail.php`).

- **database/**
Includes database migrations, factories, and seeders.
 - **migrations/**: Database migration files.
 - **factories/**: Model factories for seeding the database.
 - **seeders/**: Database seeding files.
 - **public/**
The entry point for your application. This folder contains the `index.php` file and is where assets like images, stylesheets, and JavaScript files are stored.
 - **resources/**
Contains views (Blade files), raw assets (like SASS, JavaScript), and language files.
 - **views/**: Blade templates.
 - **lang/**: Language files.
 - **routes/**
Defines your application's routes. By default, Laravel provides `web.php` (for browser-based routes) and `api.php` (for API routes).
 - **storage/**
Used to store logs, file uploads, cache, and compiled Blade templates.
 - **tests/**
Contains all your application's test files. Laravel supports PHPUnit for testing.
-

Design Patterns Used by Laravel

1. **Singleton Pattern**
Laravel's **Service Container** is a prime example of the Singleton pattern, where services are bound to the container and retrieved when needed, ensuring that only one instance of the service is used throughout the application.
2. **Facade Pattern**
Laravel uses facades like `Cache`, `DB`, and `Route` to provide an "interface" to underlying services in the service container.
3. **Observer Pattern**
Eloquent models make use of the **Observer pattern** by allowing developers to define actions when certain events occur on a model, such as `saving`, `saved`, `updating`, etc.
4. **Strategy Pattern**
The **Queue** system in Laravel is an example of the **Strategy pattern**,

as it allows you to choose from different queue drivers such as **database**, **redis**, **beanstalkd**, etc.

5. **Repository Pattern**

Laravel encourages the use of the **Repository pattern** for abstracting the logic of data access, allowing for easier testing and maintaining of the code.

Technologies Used in Laravel

1. **PHP**

Laravel is built with PHP, a widely-used server-side scripting language.

2. **Eloquent ORM**

Laravel's built-in Object-Relational Mapper (ORM) makes interacting with databases easy by allowing you to use PHP syntax to work with database records.

3. **Blade Templating Engine**

Blade is Laravel's lightweight, powerful templating engine, providing a simple way to render views and components.

4. **Artisan CLI**

The command-line interface that allows you to perform various tasks like running migrations, generating code, and clearing caches.

5. **Composer**

Laravel uses Composer for dependency management and package installation.

6. **MySQL/PostgreSQL/SQLite/SQL Server**

Laravel supports multiple database management systems, and you can choose one based on your needs.

7. **Laravel Mix**

A tool that compiles assets such as CSS and JavaScript, allowing you to use modern front-end technologies like **SASS** and **Webpack**.

8. **Testing with PHPUnit**

Laravel integrates PHPUnit for unit and feature testing, making it easy to test your application.

Conclusion

Laravel provides an elegant, clean, and simple way to handle common web development tasks. With its MVC architecture, easy-to-use Artisan CLI, and strong design patterns, Laravel simplifies and speeds up web application development.

By following best practices and using Laravel's built-in tools, you can create robust, maintainable applications quickly.
